

Trabalho 04: Sistema de Autocomplete de Jogos com Trie

Estrutura de Dados

Rafael Gontijo Ferreira

rafael.ferreira.2@fgv.edu.br

Rudá Dantas Ruoso Brandão

rudabrandao@ruoso.com

1 Descrição do Projeto

Este trabalho apresenta a implementação de um sistema de *autocomplete* para um buscador de jogos online, desenvolvido como quarta atividade da disciplina de Estrutura de Dados. O objetivo central é desenvolver um sistema capaz de receber o prefixo de um jogo e indicar quais são os jogos mais populares que começam com ele.

O sistema foi implementado via **Trie** e suporta as seguintes operações fundamentais:

- **Inserção:** Adiciona o caminho referente a um título na estrutura.
- **Busca:** Verifica se uma dada chave de busca está representada na **Trie**.
- **Recuperação:** Recuperação da lista dos jogos incluídos em uma dada sub-árvore.
- **Ordenação:** Ordena a lista por popularidade de forma decrescente; desempate por título.
- **Autocomplete:** Utiliza das operações acima para indicar quais são os jogos mais populares começam com um dado prefixo.

Representação da Trie

Tamanho do alfabeto e mapeamento de caracteres

A Trie foi implementada com um alfabeto de **36 símbolos**: as 26 letras minúsculas do alfabeto latino (a–z) mais os 10 dígitos decimais (0–9). Essa escolha cobre todos os caracteres válidos especificados pelo enunciado, sendo os espaços descartados antes da inserção.

O mapeamento de um caractere para o índice do array de filhos é realizado pelo método auxiliar `charToIndex`:

- letras minúsculas: `c - 'a'`, produzindo índices de 0 a 25;
- dígitos: `c - '0' + 26`, produzindo índices de 26 a 35.

Caracteres fora desses dois grupos retornam `-1` e são ignorados durante a travessia. Cada nó mantém um array `children[36]` inicializado com `nullptr`.

Normalização para chave de busca

O método `toSearchKey` normaliza qualquer título ou prefixo em dois passos: descarta todos os espaços em branco via `isspace` e converte cada caractere restante para minúsculo via `tolower`. Com isso, entradas como "Half Life", "halflife" e "HALF LIFE" produzem a mesma chave interna "halflife", garantindo buscas *case-insensitive* e insensíveis a espaços em todos os métodos que interagem com a **Trie**.

Funcionamento dos métodos

insert: normaliza o título do jogo para sua chave de busca e percorre a **Trie** nó a nó, criando filhos ausentes conforme necessário. Ao chegar ao nó terminal da chave, define **isEndOfTitle** = **true** e armazena o ponteiro para o objeto **Game** já existente no array da base de dados, sem criar cópias.

contains: normaliza o título recebido e tenta percorrer o caminho correspondente na **Trie**. Retorna **true** apenas se todo o caminho existe e o nó final tem **isEndOfTitle** = **true**. Qualquer filho ausente interrompe a busca e retorna **false** imediatamente.

autocomplete: normaliza o prefixo e navega até o nó correspondente na **Trie**. A partir daí, o método auxiliar **collectGamesSubtree** realiza uma busca em profundidade (DFS), varrendo todos os filhos em ordem de índice e coletando ponteiros de jogos nos nós terminais. O vetor coletado é então ordenado por **sortResults** e truncado aos k primeiros elementos com **resize**.

sortResults: ordena o vetor de ponteiros via *insertion sort*. O critério primário é popularidade decrescente; em empate, utiliza-se a chave de busca do título, obtida chamando **toSearchKey** sobre o título original, em ordem lexicográfica crescente. Essa abordagem corrige uma limitação presente em versões anteriores que comparavam o título bruto, sem normalização.

selectK: alternativa ao par **sortResults** + **resize**. Executa k passagens sobre o vetor, a cada passagem promovendo o elemento de maior popularidade (com desempate por chave de busca) para a posição i . Retorna apenas os k selecionados. O custo é $O(k \cdot r)$, vantajoso quando $k \ll r$. O método está implementado mas, na versão atual, **autocomplete** utiliza **sortResults** seguido de **resize**, mantendo as assinaturas de função exigidas e deixando **selectK** disponível como alternativa.

Análise de custo

Consideram-se os parâmetros n (total de jogos), L (comprimento da chave do título), p (comprimento da chave do prefixo), s (nós visitados na subárvore do prefixo) e r (jogos encontrados para o prefixo).

Operação	Complexidade	Origem do custo
insert	$O(L)$	Percurso de L nós na Trie
contains	$O(L)$	Percurso de L nós na Trie
autocomplete	$O(p + s + r^2)$	Navegação (p), DFS (s), insertion sort (r^2)
sortResults	$O(r^2)$	Implementado via insertion sort
selectK	$O(k \cdot r)$	k passagens lineares sobre r candidatos

A construção da **Trie**, feita uma única vez, custa $O(n \cdot L)$. O custo de cada consulta é dominado pela DFS (s nós) e pela ordenação de r resultados. Como $r \leq s \leq$ total de nós da **Trie**, e prefixos mais longos reduzem s drasticamente, o sistema é muito eficiente para buscas específicas. Para cenários com $k \ll r$, **selectK** reduz o custo de ordenação de $O(r^2)$ para $O(k \cdot r)$.

Comparação, memória e compartilhamento de prefixos

Solução ingênua: percorrer o array de n jogos a cada consulta, verificando se o título começa com o prefixo após normalização. O custo por consulta é $O(n \cdot L)$, independente do prefixo, tornando-se proibitivo à medida que a base cresce.

Com a Trie: após a inserção em $O(n \cdot L)$, cada consulta custa $O(p + s + r^2)$, com s tipicamente muito menor que n para prefixos específicos. O ganho cresce com o tamanho da base e com a especificidade do

prefixo.

Custo de memória: cada nó aloca um array fixo de 36 ponteiros (288 bytes em plataformas de 64 bits), independentemente de quantos filhos estão ocupados. Para uma base esparsa, o consumo total pode superar o custo de simplesmente armazenar os títulos. O **compartilhamento de prefixos** mitiga esse custo: títulos com início comum, como *Hades*, *Halo* e *Half Life*, compartilham os nós **h** e **a**, evitando duplicação. O tamanho do alfabeto determina diretamente o tamanho de cada nó: reduzir o alfabeto diminuiria o consumo por nó, mas exigiria um mapeamento de caracteres mais elaborado.